Pràctica 2

Objectiu

L'objectiu d'aquesta pràctica és aprendre conceptes bàsics relacionats amb la programació concurrent, usant múltiples processos i múltiples fils d'execució (multithreading).

Grups de pràctiques

Els grups de pràctiques seran de dos o tres alumnes del mateix grup de laboratori.

Entorn de treball

Les implementacions s'han d'executar sobre la distribució milax dels laboratoris de l'assignatura.

Lliuraments

Es faran dos lliuraments de la pràctica al moodle. El primer correspondrà a les fases 1 i 2 (processos), i el segon a les fases 3 i 4 (processos + threads). Heu de seguir estrictament totes les instruccions especificades a les tasques del moodle. La data límit de lliurament en primera convocatòria és la mateixa per a tots els grups de laboratori (veure l'entrada Distribució de classes dins l'espai Moodle de l'assignatura).

Correcció

La revisió i correcció de la pràctica 2 es realitzarà mitjançant un examen individual que consistirà en una modificació i execució de la pràctica. Les dates de l'examen s'especificaran en l'espai moodle de l'assignatura, així com tot els detalls corresponents a l'avaluació de la pràctica.

Es realitzarà una verificació automàtica de còpies entre tots els programes presentats. Si es detecta alguna còpia, tots els alumnes involucrats (els qui han copiat i els que s'han deixat copiar) tindran la pràctica suspesa i, per extensió, l'assignatura suspesa.

Enunciat

Es tracta d'implementar una versió bàsica del joc Arkanoid o Breakout. Es genera un camp de joc rectangular amb una porteria/sortida, una paleta que es mou amb el teclat per a cobrir la sortida i una pilota que rebotarà contra les parets del camp, contra la paleta i contra alguns blocs. Hi ha 3 tipus de blocs: uns blocs que es trenquen i desapareixen (A), uns blocs que es trenquen, desapareixen i creen una nova pilota (B) i uns blocs que no es trenquen i simplement fan rebotar la pilota. En el cas de tenir varies pilotes, al xocar entre elles hauran de rebotar. Quan una de les pilotes surti per la porteria, desapareixerà.

El programa acabarà la seva execució quan no hi hagi cap pilota al camp, o quan no quedin blocs dels que es trenquen o quan l'usuari premi la tecla RETURN. El joc es guanya si s'han trencat tots els blocs trencables, i hi ha com a mínim una pilota al camp de joc.

En el moodle us proporcionem una versió inicial del programa 'mur0.c' on de manera seqüència es controla una pilota i la paleta d'usuari. Per tal d'aprendre programació concurrent, es demana modificar aquesta versió inicial de forma que l'ordinador pugui moure més d'una pilota simultàniament. La idea principal és que les funcions que controlen el moviment de la pilota i de la paleta s'adaptin per a ser executats de manera concurrent, creant múltiples processos. Així, en

l'evolució del joc només caldrà crear tants processos com pilotes s'hagin de crear en trencar-se els blocs.

Fases de la pràctica

Per tal de facilitar el disseny de la pràctica es proposen 4 fases:

0. Versió seqüencial bàsica ('mur0.c'):

Implementació inicial de caràcter seqüencial per controlar una única pilota i la paleta de l'usuari. Es genera un camp de joc rectangular mitjançant la llibreria `winsuport2` i `curses`, on la pilota rebota contra parets, la paleta i diferents tipus de blocs. Les dimensions del tauler, la mida de la porteria i la configuració inicial de la pilota (posició i velocitat) es carreguen dinàmicament des d'un fitxer de paràmetres extern.

1. Creació del procés pilota ('mur1.c', 'pilota1.c'):

Modificació de la versió inicial per permetre l'execució concurrent mitjançant la creació de múltiples processos. El programa principal delega el moviment de la pilota a un procés independent creat mitjançant `fork()` i `execlp()` , i l'ús de memòria compartida, necessària per l'ús de la llibreria gràfica de curses. La informació que necessitem la passarem per la línia de comandes. S'implementa l'aparició dinàmica de noves pilotes en destruir blocs del tipus 'B'. En aquesta etapa s'observen deficiències visuals a la pantalla (parpellejos i caràcters sobreposats) a causa de la manca de sincronització en l'accés a la memòria compartida.

2. Sincronització de processos ('mur2.c', 'pilota2.c'):

Protecció dels recursos compartits per eliminar les inconsistències gràfiques i garantir la integritat de les dades. S'utilitza un semàfor configurat com a **mutex** per protegir les seccions crítiques d'accés al tauler (memòria compartida) i a les variables globals de recompte. Es centralitza la creació de processos al pare mitjançant una **cua de missatges** (bústia), on les pilotes demanen la creació de nous fills de forma asíncrona. Finalment, es garanteix l'alliberament net dels recursos IPC (`semàfors` i `bústies`) en acabar el joc.

3. Fils d'execució ('mur3.c', 'pilota3.c'):

Evolució del procés principal cap a una arquitectura **multifil** (*multithreaded*) i multiprocés. Es mantenen les pilotes com a processos, però s'incorporà la paleta implementada com a fil (`pthreads`) dins de `mur3.c`.

Configuració Dinàmica: El programa llegeix del fitxer un nombre variable de paletes amb posició (x,y), velocitat lateral i **mida específica**. Aquestes noves paletes no llegiran del teclat, i el seu desplaçament horitzontal serà automàtic amb la velocitat lateral fixada pel fitxer. Hi haurà un màxim de 9 paletes autònomes, i portaran estaran representades per un nombre del 1 al 9.

Interacció Reactiva: Mitjançant les tecles numèriques (1-9), el jugador pot activar un desplaçament vertical de la paleta corresponent, la qual es desplaçarà una fila mentre no trobi cap obstacle. Al trobar un obstacle (bloc, pilota, paleta) passarà a canviar el sentit de desplaçament.

4. Interacció complexa i super-pilotes ('mur4.c', 'pilota4.c'):

Implementació d'una lògica de joc avançada amb poders especials i comunicació bidireccional. S'introdueixen els blocs 'T' que activen **super-poders temporals** (dibuix invers i destrucció de murs irrompibles) gestionats per un temporitzador global a la memòria compartida.

Comunicació entre Pilotes: En sortir per la porteria, una pilota envia la seva velocitat a la bústia per influir en el moviment de les pilotes restants.

Threads interns a les pilotes: Cada procés pilota esdevé multifil, creant un **fil receptor** per escoltar la bústia sense bloquejar el moviment principal. S'utilitzen **mutexes dels pthreads** (`pthread_mutex_t`) per a crear seccions crítiques entre els threads de cada pilota.

Fase 0

Per tal de generar la visualització del joc es proporcionen una sèrie de rutines dins del fitxer **'winsuport2.c'**, que utilitzen la llibreria d'UNIX **'curses'**. Aquesta versió de la llibreria està dissenyada per treballar amb memòria compartida o fils, separant la memòria del tauler de la visualització física a la pantalla. El fitxer **'winsuport2.h'** inclou les definicions i les capçaleres de les rutines implementades:

```
int  win_ini(int *fil, int *col, char creq, unsigned int inv);
void win_set(void *ptr, int fil, int col);
void win_update(void);
void win_fi(void);
void win_escricar(int f, int c, char car, unsigned int invers);
char win_quincar(int f, int c);
void win_escristr(char *str);
int  win_gettec(void);
void win_retard(int ms);
```

`win_ini`: S'ha d'invocar abans d'utilitzar qualsevol altra funció. Indica el número de files i columnes del camp de joc, el caràcter del marc i si es visualitza en atribut invers. Retorna el **número de bytes de memòria** que cal reservar per al tauler o un valor negatiu si hi ha un error. L'última fila es reserva per a missatges del sistema.

`win_set`: S'utilitza per vincular el punter de la memòria (ja sigui local o compartida) amb la llibreria. Cal cridar-la després d'obtenir el punter de la memòria i abans d'intentar dibuixar res.

`win_update`: Atès que `winsuport2` treballa sobre un tauler en memòria, els canvis fets amb `win_escricar` no són visibles immediatament. Cal invocar aquesta funció des del procés o fil principal per **bolcar el contingut de la memòria a la pantalla** física.

`win_fi`: S'ha d'invocar en acabar el programa per tancar correctament l'entorn de les `curses`.

`win_escricar`: Escriu un caràcter en una posició de la memòria del tauler. Permet ressaltar el caràcter amb el paràmetre `invers`. S'utilitza per identificar els elements del joc (paret `+`, paleta `0`, pilota `1`, etc.).

`win_quincar`: Permet consultar el caràcter ASCII que hi ha actualment en una posició determinada de la memòria.

`win_escristr`: Escriu una cadena de caràcters a l'última línia de la finestra, útil per mostrar el temps o l'estat del joc.

`win_gettec`: Llegeix una tecla del teclat. Es recomana utilitzar les definicions de tecles de `winsuport2.h` per controlar el moviment de la paleta.

`win_retard`: Atura l'execució del procés o fil durant el temps especificat en mil·lisegons per controlar la velocitat del joc

Execució i Configuració del Mur0

El programa d'exemple **'mur0.c'** utilitza les funcions de la llibreria **`winsuport2`** per dibuixar el camp de joc i controlar de manera seqüencial el moviment de la paleta i de la pilota. Per generar l'executable, cal utilitzar el fitxer Makefile per compilar i enllaçar els fitxers font amb la llibreria de `curses` i el gestor de memòria:

```
$ make mur0
```

Paràmetres de Línia de Comandes

Abans d'executar-lo, cal conèixer els arguments que el programa espera per línia de comandes. El primer argument és obligatori i indica el fitxer de text amb la configuració inicial. L'estructura del fitxer s'ha adaptat per incloure la definició de la paleta:

Línia 1: `num_files num_columnes mida_porteria` (números enters)

Línia 2: `pos_columna_paleta mida_paleta` (números enters).

Línia 3: `pos_fila_pil pos_col_pil vel_fila_pil vel_col_pil` (números reals).

Per exemple, un fitxer **'params.txt'** estàndard contindria:

```
25 40 20
15 8
21.0 20.0 -1.0 0.3
```

Aquest fitxer indica una zona de dibuix de 25 files i 40 columnes (camp de joc de 24x40) amb una porteria de 20 espais. La paleta se situarà inicialment a la columna 15 amb una mida de 8 caràcters. La pilota començarà a la posició (21, 20) amb una velocitat de -1.0 (cap amunt) i 0.3 (cap a la dreta).

Retard i Invocació

Es pot afegir un segon argument opcional per indicar el **retard general** del moviment en mil·lisegons. Si no s'especifica, el valor per defecte és de **100 ms**. La sintaxi d'invocació és:

```
$ ./mur0 fit_config [retard]
```

Exemple d'execució amb 50 mil·lisegons de retard:

```
$ ./mur0 params.txt 50
```

[!IMPORTANT] Redimensioneu el terminal amb prou files i columnes abans de començar per evitar errors en la inicialització de les `curses`. Si s'executa el programa sense arguments, es mostrarà un missatge d'ajuda

Fase 1: Creació del procés pilota ('mur1.c', 'pilota1.c')

En aquesta fase, l'objectiu és modificar la lògica seqüencial per tal que el moviment de la pilota s'executi com a un **procés independent** del programa principal. A diferència de la Fase 0, on tot s'executava en un sol flux, ara utilitzarem les funcions del sistema per crear una execució concurrent mitjançant un procés pare (gestor) i un o més processos fills (pilotes).

Per fer-ho, dividirem el codi en dos executables separats: **`mur1`** (el gestor del joc i la paleta) i **`pilota1`** (el programa que controla el moviment de cada pilota).

```
$ cp mur0.c mur1.c
```

Caldrà crear també el fitxer pilota1.c per al nou executable.

Pas 1.1: Anàlisi de l'estructura i variables

Abans de realitzar la separació, cal entendre com es distribueixen les dades. En utilitzar **`winsuport2`**, el tauler de joc ja no resideix en una variable local simple, sinó en un bloc de memòria al qual ambdós processos hauran de tenir accés.

Les variables globals clau que heretem del `mur0.c` són:

`int n_fil, n_col;` (dimensions del taulell).

`int m_por;` (mida de la porteria).

`int f_pal, c_pal, m_pal;` (dades de la paleta).

`float pos_f, pos_c, vel_f, vel_c;` (estat de la pilota).

Pas 1.2: Reestructuració de les funcions

El programa s'organitza ara de manera que les responsabilitats quedin dividides:

`int carrega_configuracio(FILE *fit);`: Segueix al pare per llegir els paràmetres inicials.

`int inicialitza_joc(void);`: El pare reserva la memòria compartida i dibuixa el mur inicial.

`int mou_pilota(void);`: Aquesta funció es trasllada a **`pilota1.c`**, on s'executarà en un bucle infinit dins del procés fill. També es traslladaran totes les funcions a les que fa referència el moviment de la pilota.

`int mou_paleta(void);`: Es manté al procés pare (**`mur1.c`**) per permetre al jugador controlar el joc.

Estructura del programa principal (mur1.c)

El bucle principal del pare ja no crida directament al moviment de la pilota. En el seu lloc, crea el procés fill i es dedica a actualitzar la pantalla i gestionar la paleta:

```
int main(int n_args, char *ll_args[])
{
  /* 1. Inicialització de memòria compartida i curses */
  if (inicialitza_joc() != 0) exit(4);

  /* 2. Creació del procés fill per a la pilota */
  if (fork() == 0)
  {
      /* Execució de ./pilota1 passant id_mem, posició i velocitat per argv */
      execlp("./pilota1", "pilota1", ..., (char *)NULL);
      exit(1);
  }

  /* 3. Bucle de gestió (Pare) */
  do
  {
      fi1 = mou_paleta();        /* Gestió del teclat */
      actualitza_temps();        /* Control de minuts:segons */
      win_update();              /* Refresc visual (propi de winsuport2) */
      win_retard(retard);
  } while (!fi1 && !fi2);

  win_fi();
}
```

Nota sobre el sincronisme: En aquesta fase, com que no hi ha semàfors, observareu que la pilota i la paleta poden "esborrar-se" mútuament o produir parpellejos, ja que ambdós processos accedeixen a la memòria compartida simultàniament sense control

Creació del procés pilota (‘mur1.c’, ‘pilota1.c’)

Partint de l’exemple anterior, modifiqueu les funcions que controlen el moviment de la pilota per a què pugui actuar com a un procés independent.

Feu una còpia del codi abans de començar:

```
$ cp mur0.c mur1.c
```

Pas 1.1: Creació del programa de la Pilota (`pilota1.c`)

Per tal que la pilota es mogui de forma concurrent, crearem un nou fitxer codi font anomenat `pilota1.c` que compilarà com a un executable separat.

Tasques:

Aquest programa necessitarà un `main()` propi.

Com que la memòria compartida ja està creada pel procés principal, `pilota1` ha de rebre per arguments de la línia de comandes (via `argv`) la informació necessària: l'identificador de la memòria compartida (`id_mem`), les dimensions del tauler, la posició inicial de la pilota, la velocitat i el retard.

Un cop convertit l'`id_mem` a enter, el procés ha de cridar `map_mem(id_mem)` per obtenir el punter, i connectar-se al tauler amb `win_set(punter, files, columnes)`.

Moveu la funció `mou_pilota()` a aquest fitxer. Ara el bucle principal de `pilota1` només consistirà a moure la pilota i fer un retard (`win_retard()`). *Atenció: el procés pilota no ha de cridar `win_update()` ni llegir el teclat!*

Pas 1.2: Modificació del programa principal (`mur1.c`) i control del temps

Ara el programa principal assumirà el rol de gestor del joc i de la paleta.

Tasques a la funció `main`:

Creació del procés pilota: Després d'inicialitzar la memòria compartida i el tauler, feu un `fork()` i un `exec()` (per exemple, `execlp`) per posar en marxa l'executable `./pilota1`. Passeu-li tots els arguments necessaris com a text (usant `sprintf`).

Control del temps i bucle principal: El bucle d'aquest procés pare ara només ha d'invocar la funció de moviment de la paleta i `win_update()`. A més, **haurà de controlar el temps de joc (minuts:segons)** i mostrar-lo contínuament per la línia de missatges (usant `win_escristr()`), fins que es doni alguna condició de finalització.

Missatges de finalització: L'última acció del programa, just abans de tancar, serà la d'escriure el temps total de joc per la sortida estàndard (amb un `printf`), juntament amb els missatges de finalització proposats en la versió d'exemple original.

Proveu els canvis generant els executables (per exemple `$ make mur1`).

ATENCIÓ: podria ser que la pilota es mogués molt ràpid i no la vegéssiu. Assegureu-vos d'ajustar el paràmetre de retard al llançar el programa.

Pas 1.3: Dinàmica de col·lisions i aparició de noves pilotes (B)

Un cop la primera pilota es mou correctament, el següent objectiu és gestionar la destrucció de blocs. Cal modificar la lògica de col·lisions a **`pilota1.c`** per detectar l'impacte amb blocs de tipus **'B'**. Quan això passi:

Creació concurrent: El propi procés pilota invocarà un `fork()` i un `execlp()` per instanciar un nou executable `./pilota1`.

Estat inicial: La nova pilota s'ha de crear en la mateixa posició del bloc destruït, però invertint la seva velocitat perquè realitzi l'efecte de rebot immediat.

Nota sobre el rendiment visual: En aquesta fase, com que encara no utilitzem semàfors, és totalment normal observar "problemes en l'entorn gràfic": caràcters que desapareixen, parets que s'esborren parcialment o pilotes que "mengen" trossos de la paleta. Aquests són els conflictes clàssics de la memòria compartida sense sincronització.

Pas 1.4: El gestor del joc (mur1.c) i el control del cronòmetre

En aquesta fase, el programa principal es converteix en el **mestre de cerimònies**. Ja no mou la pilota directament, sinó que gestiona l'entorn:

Instanciació inicial: Abans d'entrar al bucle, el pare fa el primer `fork()` per llançar la pilota original, passant-li l'identificador de la memòria compartida (`id_mem`) i els paràmetres inicials via `argv`.

Bucle de refresc: El bucle del pare ara és més lleuger. Només ha d'executar `mou_paleta()` per recollir el *input* de l'usuari i, molt important, cridar a **`win_update()`** per bolcar els canvis de la memòria a la pantalla física.

Control del temps: Cal implementar un cronòmetre en format `minuts:segons`. Com que coneixem el valor de `retard`, podem usar un comptador intern, **no** s'han d'usar les rutines de la llibreria Time.h. Aquest temps s'ha d'imprimir a la part inferior del tauler amb `win_escristr()`.

Finalització: Quan es compleixi una condició de final (no queden blocs, no queden pilotes o es prem `RETURN`), el programa tancarà les `curses` i imprimirà per la sortida estàndard (`printf`) el temps total de la partida i el resultat del joc.

Pas 1.5: Independència de dades i gestió local del procés

És vital que cada procés pilota gestioni la seva pròpia "personalitat". Tot i que el tauler de joc és un recurs compartit a la memòria IPC, dades com la **posició exacta (float)** i la **velocitat actual** han de ser variables locals dins de cada procés fill.

Identificació: En crear una nova pilota des de `pilota1.c`, li passarem un caràcter identificador (per exemple, 'a', 'b', etc.) per poder-les distingir visualment.

Localitat vs. Globalitat: Si féssim servir la memòria compartida per a la velocitat de totes les pilotes, totes es mourien alhora cap a la mateixa direcció. La gràcia dels processos independents és que cadascú té les seves pròpies dades privades, garantint un moviment totalment autònom.

Consell per al laboratori: Si en fer `make mur1` veieu que la pilota va "a la velocitat de la llum", reviseu que el procés fill estigui executant el seu propi `win_retard()` dins del bucle de `pilota1.c`.

Fase 2. Sincronització de processos ('mur2.c', 'pilota2.c')

L'objectiu d'aquesta fase és resoldre els defectes visuals i les inconsistències de dades observades en la fase anterior. Aquests problemes (parpellejos, caràcters sobreposats o esborrats incorrectes) es deuen a l'accés concurrent desordenat de diversos processos a la memòria compartida. Per solucionar-ho, s'implementaran mecanismes de sincronització mitjançant les llibreries `semafor.h` i `missatge.h`.

Pas 2.1: Identificació de les Seccions Crítiques

Abans de programar, cal identificar els recursos compartits que requereixen accés exclusiu per evitar condicions de carrera:

- **El tauler (Memòria compartida):** La seqüència de consultar una casella (`win_quincar`) i actualitzar-la (`win_escricar`) ha de ser atòmica; cap procés ha d'intervenir entremig.
- **Variables globals compartides:** Qualsevol comptador situat a la memòria compartida (com el nombre de blocs o pilotes) que sigui llegit i modificat per múltiples processos ha de ser protegit.
- **Rutines no reentrants:** Aquelles funcions que utilitzen variables globals internes (totes les rutines de les curses, excepte win_retard) si es criden de manera concurrent han de garantir l'exclusió mútua.

Pas 2.2: Creació del semàfor per exclusió mútua (Mutex)

Es farà servir un semàfor inicialitzat a 1 per actuar com a **secció crítica**.

1. **Inicialització:** Al programa principal (`mur2.c`), s'ha d'invocar `ini_sem(1)` abans de crear cap procés fill per obtenir l'identificador `id_sem`.
2. **Comunicació d'IDs:** L'identificador `id_sem` s'ha de passar com a argument de text a tots els processos pilota mitjançant `execlp()`, tal com es va fer amb la memòria compartida.

Pas 2.3: Protecció de l'accés concurrent

Un cop els processos coneixen `id_sem`, s'aplica el protocol d'exclusió mútua:

1. **Entrada a Secció Crítica:** Abans de calcular rebots o modificar el tauler, cada procés ha d'executar `waitS(id_sem)`. Si el recurs està ocupat, el procés es quedarà bloquejat fins que s'alliberi el recurs.
2. **Sortida de Secció Crítica:** Immediatament després d'actualitzar la memòria, s'ha d'executar `signalS(id_sem)` per permetre l'accés a altres processos.
3. **Les seccions crítiques** s'han de fer el més petit possible, per tal d'executar els diferents processos amb la màxima concurrència possible.

Pas 2.4: Comunicació asíncrona mitjançant Missatges

Per evitar la creació de processos orfes o problemes de jerarquia, el pare (`mur2.c`) serà l'únic encarregat de fer `fork()`. S'ha de modificar el comportament de la fase anterior per obtenir aquest comportament.

1. **Cua de Missatges:** El pare inicialitza la cua amb `ini_mis()` i passa l'identificador `id_mis` a les pilotes.
2. **Petició de creació:** Quan una pilota trenca un bloc 'B', envia un missatge asíncron al pare amb `sendM()` contenint les coordenades i velocitats de la nova pilota.

3. **Lectura no bloquejant:** Atès que `receiveM()` és bloquejant per defecte, s'utilitzarà una estratègia per evitar que el pare es congeli. Una primera solució és que el pare s'enviï un missatge de control a si mateix: si el primer missatge que llegeix és el seu, vol dir que la cua no tenia peticions de pilotes noves, s'han de llegir els missatges fins arribar al missatge que s'ha autoenviat el pare. Totes les operacions sobre la bústia són atòmiques, no cal seccions crítiques per usar-les.

Pas 2.5: Alliberament de recursos IPC

Els recursos IPC no s'alliberen automàticament en finalitzar els processos.

1. **Neteja programada:** En acabar el joc, el procés principal ha d'executar obligatòriament `elim_mem()`, `elim_sem()` i `elim_mis()`.
2. **Gestió d'errors:** Si el programa finalitza bruscament, cal netejar manualment el sistema amb l'script `./mataipc` o les comandes `ipcs` (per llistar) i `ipcrm` (per eliminar) des del terminal.

3. Fase 3: Arquitectura híbrida amb Fils d'Execució ('mur3.c')

Fins ara, el sistema ha treballat exclusivament amb processos independents, una aproximació robusta però que requereix mecanismes complexos (IPC) per compartir dades. En aquesta fase, mantindrem els processos per a les pilotes, però transformarem el procés principal en un **multifil** (*multithreaded*). L'objectiu és fer que la paleta sigui un thread i afegir fins a 9 **paletes autònomes** que actuaran també com a fils (`pthreads`) dins del procés mur.

Preparació:

```
$ cp mur2.c mur3.c
# No cal copiar pilota2.c, ja que els processos pilota no es modifiquen.
```

Pas 3.1: Configuració Dinàmica i Mida Variable

A diferència de les fases anteriors, el nombre de paletes no serà fix. S'ha de modificar la funció de càrrega per detectar línies que comencin pel caràcter **'P'** al fitxer de paràmetres.

- **Dades per línia:** Cal llegir la fila, la columna inicial, la direcció lateral i la **mida (amplada)** de la paleta.

Pas 3.2: La funció del fil (Salt i Moviment Autònom)

La llibreria `pthread` requereix funcions amb la signatura `void * funcio(void * arg)`. Cal incloure `#include <pthread.h>` i afegir l'opció `-lpthread` al Makefile.

- **Moviment Lateral:** Cada fil mourà la seva paleta horitzontalment de forma constant segons la velocitat rebuda pel fitxer de configuració, rebotant horitzontalment en trobar obstacles o els límits del tauler.
- **Mecànica del moviment vertical:** Quan el fil detecti que l'usuari ha premut la tecla corresponent al seu numero, es mourà una posició cap a dalt o cap a baix. si al intentar moure's xoca amb algun element, canviarà el sentit de desplaçament vertical.
- **Gestió de Mida:** Totes les operacions d'esborrat, dibuix i col·lisió han de gestionar el bloc complet de caràcters segons la mida definida per a cada paleta.

Pas 3.3: Sincronització IPC Híbrida

Aquest és el punt crític: les paletes autònomes (fils) interaccionen amb el mateix tauler de memòria compartida que les pilotes (processos).

- **Semàfor compartit:** Dins del fil, abans de qualsevol crida a `win_quincar` o `win_escricar`, s'ha d'utilitzar exactament el **mateix semàfor IPC** que fan servir els processos.
- **Exclusió mútua:** Les crides a `waitS(id_sem)` i `signalS(id_sem)` garanteixen que cap fil o procés modifiqui el tauler mentre un altre l'està llegint o actualitzant.

Fase 4. Interacció complexa i super-pilotes ('mur4.c', 'pilota4.c')

En aquesta darrera fase s'apliquen de forma integrada tots els mecanismes de comunicació i sincronització estudiats per implementar una lògica de joc interactiva i dinàmica. L'objectiu és gestionar poders temporals i la comunicació directa entre els diferents processos i fils del sistema.

Pas 4.1: Blocs especials i temporitzador en memòria compartida

Es modifica el tauler substituint els blocs destructibles ('A') per un nou tipus de bloc especial (**'T'**) que atorga poders temporals a les pilotes durant un període de temps pre-establert (`temps_poder`).

Pas 4.2: Activació de super-poders i destrucció de murs

Quan una pilota impacta contra un bloc 'T', el temporitzador s'incrementarà en 5 segons (acumulatius si el poder ja estava actiu). Mentre el `temps_poder` sigui superior a 0:

- **Efecte visual:** Les pilotes es dibuixen en mode **INVERS** per indicar l'estat especial.
- **Capacitat de destrucció:** Les pilotes guanyen la capacitat de destruir parcialment els murs irrompibles (`###` o `WLLCHAR`) en impactar-hi, a més de realitzar el rebot habitual.

Pas 4.3: Rol del pare en el control del temporitzador

El procés principal (`mur4.c`) assumeix la responsabilitat de gestionar el compte enrere dels poders.

- **Increment o Decrement:** Dins del seu bucle, el pare decrementa la variable `temps_poder` segon a segon fins a arribar a 0. El pare serà l'únic que ha de veure el valor del temporitzador. Les pilotes seran les que avisaran de l'increment i el pare ho portarà a terme. Es a dir, aquesta variable **no** ha de ser global.
- **Visualització d'estat:** S'ha de mostrar de forma combinada el temps total de partida i el temps restant de les "SUPER-PILOTES" a la línia de missatges inferior.

Pas 4.4: Comunicació de dades entre pilotes (Bústia)

S'implementa un mecanisme de cooperació on les pilotes que surten per la porteria influeixen en les que queden vives.

- **Sacrifici de la pilota:** Just abans de finalitzar, la pilota envia la seva velocitat vertical a la bústia de missatges (`sendM`) per a cadascuna de les pilotes restants.
- **Algorisme d'ajust:** En rebre la velocitat d'una pilota "sacrificada", el fil receptor incrementa o decrementa la velocitat de la seva pilota accelerant-la o alentint-la segons el valor rebut, respectant sempre els límits de velocitat establerts .

Pas 4.5: Sincronització entre els fils d'un mateix procés.

Atès que cada procés pot disposar de varis fils d'execució, cal garantir la coherència interna .

- **Mutex POSIX:** S'utilitza un `pthread_mutex_t` sempre que sigui possible.
- **Reflexió final:** Aquesta fase il·lustra la convivència de **semàfors IPC** per a la sincronització global entre processos i **mutexes de pthreads** per a la sincronització interna de cada procés multifil